ROLLUPOS V1.2 • OVER-THE-AIR (OTA) TELEMETRY & CACHE GOVERNANCE

Automatic App Update Detection & Out-of-Date Notification System

Deterministic SHA-256 Build Hashing • Service Worker Cache Bypass • Live Hub & In-App Update Indicators

1. How Users See Out-of-Date Apps (Two Live Surfaces)

A. On the RollupOS Hub Launcher (`index.html`)

- **Top OTA Banner:** Lists all authorized apps with newer builds + a 1-tap `⬆️ Update All` button.
- **App Card Badges:** Cards transition automatically between ✨ **Ready**, 🟢 **Up to Date**, and 🔴 **vOld → vNew** • ⬆️ **Out of Date**.
- **Auto-Check Triggers:** Runs on Hub launch, every **60s** while visible, on phone unlock (`visibilitychange`), and on network reconnect (`online`).

B. Inside Any Open Mini-App (`rollup-bridge.js v1.2.0`)

- **29px Docked Top System Strip:** Passively displays the active build version (`v1.3.0`) on the right side of the top strip.
- **Live In-App Alert Pill:** Polls every **45s** while open (and on phone unlock). The instant a new push lands on GitHub or LAN, an amber `⬆️ Out of Date • Update vX.Y.Z` pill lights up in the top strip.
- **1-Tap Clean Reload:** Purges stale CacheStorage entries for that app, updates the device ledger, and reloads cleanly.

2. 4-Pillar Systems Architecture & Blind Spots Solved

Blind Spot / Trap	Why Naive Version Checks Fail	Deterministic RollupOS Solution
1. Forgotten Manual Version Bumps	Developers and AI coding sessions frequently edit `apps/<id>/index.html` and push without manually incrementing `"version"` in `registry.json` .	`rollup.py stamp` & `publish` deterministically hash every file in `apps/<id>/` (CRLF-normalized SHA-256 `build_hash`) and **auto-bump the patch version** whenever content changes.
2. Service Worker Serving Stale `registry.json`	Standard PWA fetch handlers or browser HTTP caches return cached `registry.json` responses, blinding the phone to remote updates.	`sw.js` explicitly routes any `?__ota=` probe straight to `fetch(req, { cache: 'no-store' })` , and `rollup.py serve` serves live in-memory hashes with `Cache-Control: no-store` .
3. Multi-Session Git Lock Collisions	Writing hashes to `registry.json` on disk during every local HTTP poll dirties the working tree while parallel agents are coding.	Local `GET /registry.json?__ota=...` computes hashes purely **in-memory** (cached by `mtime_ns`), writing to disk only on `rollup.py stamp` or `publish` .
4. APK Wrapper vs. Web Cartridge Split	Users wonder if they must re-download `RollupOS.apk` whenever a mini-app changes.	**Zero APK reinstall required** for mini-app, Hub, or Bridge updates (100% OTA). Only native Kotlin permission changes require a new APK (`hub.apk_version`).

3. Zero-Touch Developer Workflow

- **Cloud / GitHub Pages Push:** Run `python rollup.py publish` — automatically stamps SHA-256 `build_hash` across all 7 mini-apps, auto-increments forgotten patch versions, updates `sw.js` (`CACHE_NAME = 'rollup-hub-<etag>'`), commits, and pushes.
- **Instant Local LAN Testing (`http://192.168.86.47:8090`):** Simply save any file inside `apps/<id>/` — within 45 seconds (or the instant you unlock/refocus your phone), the running app's top strip lights up with ⬆️ `Out of Date • Update` even before pushing to GitHub!